

Statistical Methodology of Stochastic Reserving

Technical Reference & Python Function Mapping

Actuarial Systems Documentation

June 1, 2026

Abstract

This document serves as a technical companion to the stochastic reserving script. It outlines the statistical theory, mathematical equations, and assumptions underlying the Basic Chain Ladder (BCL), the Mack Model, and the Bootstrap Over-Dispersed Poisson (ODP) model, mapped directly to their respective functions in the `chainladder` Python library.

1 The Basic Chain Ladder Method

The core assumption of the Chain Ladder method is that proportional developments of claims from one development period to the next are independent of the accident year. Let $C_{i,k}$ denote the cumulative claims amount for accident year i and development period k .

1.1 Age-to-Age Factors (Link Ratios)

Python Function: `triangle.link_ratio`

The algorithm calculates the volume-weighted average of the development factors, denoted as f_k . This represents the expected ratio of claims at development period $k + 1$ compared to period k :

$$\hat{f}_k = \frac{\sum_{i=1}^{n-k} C_{i,k+1}}{\sum_{i=1}^{n-k} C_{i,k}} \quad (1)$$

The Ultimate claims for an incomplete accident year i (where current development is $n + 1 - i$) are projected as:

$$\hat{C}_{i,n} = C_{i,n+1-i} \prod_{j=n+1-i}^{n-1} \hat{f}_j \quad (2)$$

2 The Mack Model (Deterministic Volatility)

While the Basic Chain Ladder provides a point estimate, Thomas Mack (1993) introduced a distribution-free stochastic model to calculate the standard error of these estimates.

2.1 Standard Error Calculation

Python Function: `cl.MackChainladder().fit(triangle)`

Mack defines the Mean Squared Error (MSE) of the reserve estimate $\hat{R}_i$ as the sum of *Process Variance* (inherent randomness of future claims) and *Parameter Estimation Error* (uncertainty in estimating f_k from limited historical data):

$$mse(\hat{R}_i) \approx \text{Process Variance} + \text{Parameter Error} \quad (3)$$

To calculate this, the Mack model first estimates the variance parameters α_k^2 for each development period:

$$\hat{\alpha}_k^2 = \frac{1}{n-k-1} \sum_{i=1}^{n-k} C_{i,k} \left(\frac{C_{i,k+1}}{C_{i,k}} - \hat{f}_k \right)^2 \quad (4)$$

These variance parameters are then used to calculate the standard error (the **Mack Std Err** column in the Python summary).

3 Stochastic Simulation: The Bootstrap ODP Model

To calculate full capital distributions and Value at Risk (VaR), we must move beyond standard errors and simulate thousands of possible future outcomes using Monte Carlo Bootstrapping.

3.1 The Over-Dispersed Poisson Assumption

Python Function: `cl.BootstrapODPSample(n_sims=10000)`

This function models the *incremental* claims $q_{i,k}$. It assumes they follow an Over-Dispersed Poisson (ODP) distribution, meaning the variance is proportional to the mean by a dispersion parameter ϕ :

$$E[q_{i,k}] = m_{i,k} \quad \text{and} \quad Var(q_{i,k}) = \phi \cdot m_{i,k} \quad (5)$$

3.2 The Bootstrapping Algorithm

The Python function executes the following loop 10,000 times:

1. **Calculate Expected Claims:** Using the standard Chain Ladder, it calculates the expected incremental claims $m_{i,k}$ for the historical triangle.
2. **Calculate Pearson Residuals:** It calculates the scaled difference between the actual historical data and the expected data:

$$r_{i,k} = \frac{q_{i,k} - m_{i,k}}{\sqrt{m_{i,k}}} \quad (6)$$

3. **Resample with Replacement:** It randomly draws residuals from this pool (with replacement) to create a new, simulated matrix of residuals $r_{i,k}^*$.
4. **Create Pseudo-Data:** It algebraically reverses the residual formula to create a completely new synthetic triangle of claims:

$$q_{i,k}^* = m_{i,k} + r_{i,k}^* \sqrt{m_{i,k}} \quad (7)$$

4 Aggregation and Value at Risk (VaR)

Python Syntax: `boot_model.ibnr_.values.sum(axis=2).flatten()`

The `cl.Chainladder().fit(simulated.triangles)` function projects all 10,000 pseudo-triangles to ultimate simultaneously, outputting a 4-dimensional hypercube with shape (*Simulations, Lines of Business, Development Period, Accident Year*).

By applying `.sum(axis=2)`, the algorithm mathematically marginalizes out the Accident Year dimension. For a given simulation s in the space of $S = 10000$:

$$\text{Total IBNR}_s = \sum_{i=1}^n IBNR_{s,i} \quad (8)$$

This collapses the hypercube into a 1-dimensional array of 10,000 independent totals.

Python Syntax: `np.percentile(clean_simulations, 95)`

Finally, calculating the Value at Risk (VaR) involves finding the order statistic. If $F_X(x)$ is the empirical cumulative distribution function of the 10,000 simulated IBNR totals, the 95th percentile VaR is the inverse CDF:

$$VaR_{95\%} = \hat{F}_X^{-1}(0.95) \quad (9)$$